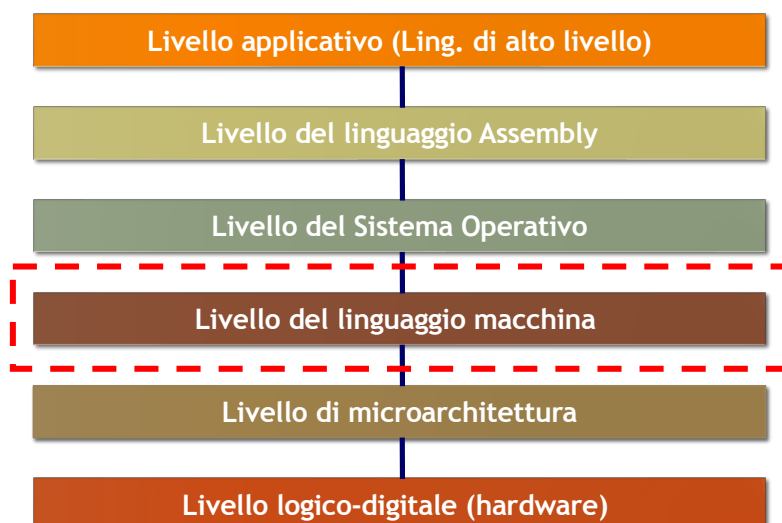


Livello del linguaggio macchina

INSTRUCTION SET
ARCHITECTURE (ISA)

Il livello del linguaggio macchina



Livello ISA

- L'**Instruction Set Architecture (ISA)** comprende gli aspetti di un elaboratore che interessano a chi progetta un compilatore (o a chi programma a basso livello)
 - è il livello di linguaggio macchina
- Un programma in linguaggio macchina si ottiene tipicamente come risultato di un processo di **traduzione**
 - può essere ottenuto anche programmando direttamente in linguaggio macchina (raro)

ISA

- Il livello ISA specifica:
 - Modello di memoria
 - Registri visibili
 - Tipi di dati
 - Modalità di indirizzamento
 - Set di istruzioni & Controllo del flusso di esecuzione
- Caratteristiche di progettazione del livello ISA:
 - Implementazione efficiente
 - Completezza
 - Durata nel tempo
 - Retrocompatibilità

Livello ISA

- Le caratteristiche computazionali di un elaboratore dipendono in gran parte dal livello ISA
- Diversi processori sono **compatibili** tra loro se si basano sullo stesso livello ISA
- **Famiglia di processori**: diverse implementazioni dello stesso livello ISA
 - Se un programma può essere eseguito da un processore della famiglia, allora può essere eseguito anche da quelli più potenti senza necessità di essere modificato
 - Retrocompatibilità

7

Famiglia dei processori INTEL

1975

2002

8080 - **8086** - 80286 - 80386 - 80486 - Pentium - Pentium II-III-IV - - - **Itanium**

Caratteristiche	8086/8088	80286	Pentium	Pentium III
Address bus	20 bit	24 bit	32 bit	36 bit
Data bus	16 / 8 bit	16 bit	64 bit	64 bit
Indirizzamento	1M byte	16M byte	4G byte	64G byte
Registri	16 bit	16 bit	32 bit	32 bit

L'architettura dei processori Intel è un'**evoluzione** e non una **sostituzione** di quella del processore 8086

8

Perché studiare l'8086?

- Molte CPU moderne compatibili con l'8086
 - Set di istruzioni mantenuto e ampliato
 - Metodi di indirizzamento più evoluti
- Set di istruzioni CISC semplice
 - Adatto a scopi didattici
- Caratteristiche principali
 - address bus: 20 bit → memoria indirizzabile: 1MB
 - data bus: 16 bit
 - 14 registri da 16 bit
 - 7 metodi di indirizzamento

Livello del linguaggio macchina

ORGANIZZAZIONE DELLA
MEMORIA
ORGANIZZAZIONE DEI REGISTRI

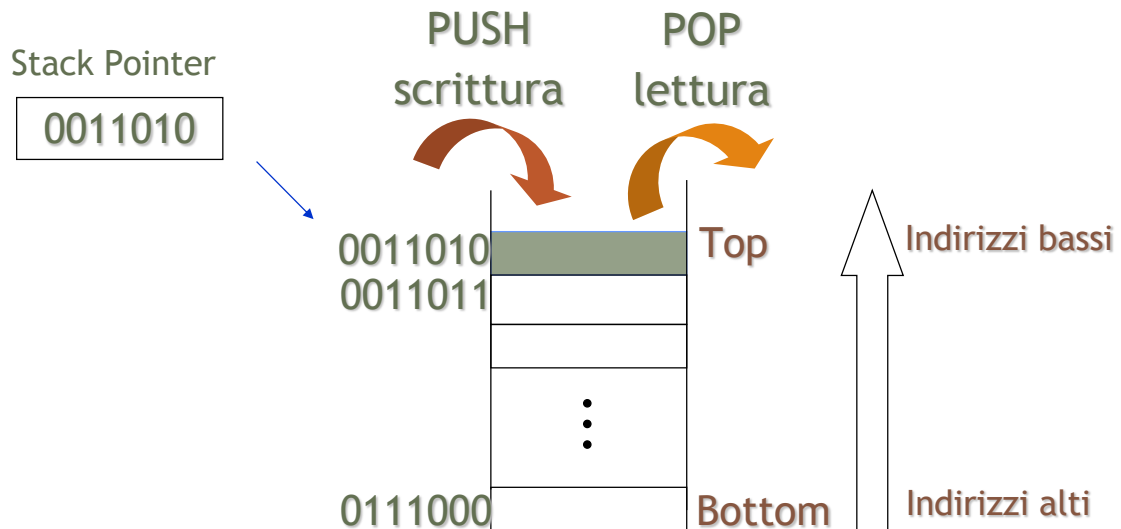
Riferimenti alle informazioni

- In linguaggio macchina è possibile **riferirsi** a un dato specificandone la **posizione**
- Un dato può trovarsi in:
 - Memoria centrale
 - Registro
 - Stack

Lo stack

- Lo stack è un insieme di locazioni contigue di memoria centrale in cui l'accesso è consentito solo all'ultimo dato immagazzinato
 - Accesso **LIFO** (Last In First Out)
- Si può leggere un dato solo dall'ultima locazione scritta (**top dello stack**)
- Si può scrivere un dato solo nella locazione successiva al top dello stack

Lo Stack



20

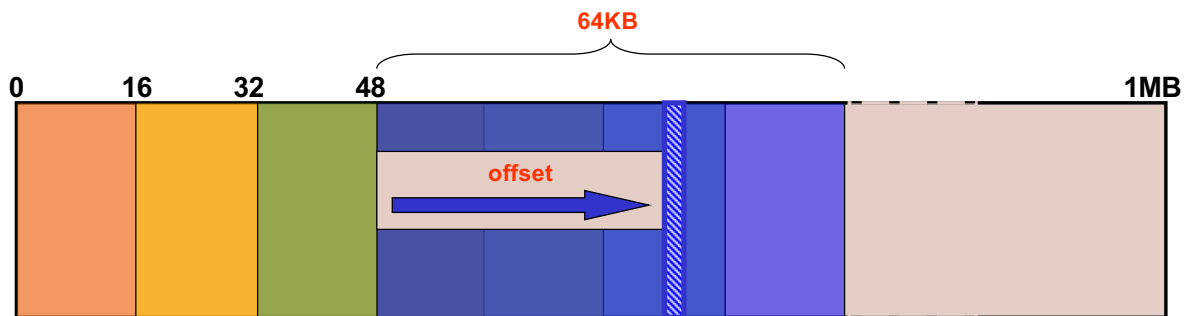
Organizzazione della memoria (8086)

- Dimensione di una locazione: **8 bit** (= 1 byte)
- Lunghezza registri: **16 bit** → 65536 (64K) locazioni
- In realtà: **1MB** di memoria indirizzabile
 - Indirizzi numerati da 0 a 1.048.575 ($= 2^{20} - 1$)
 - In esadecimale da 00000 a FFFFF (5 cifre)
 - In binario: da 00000000...0 a 11111111...1 (20 bit)
- La memoria è **segmentata**, ossia suddivisa in blocchi di locazioni consecutive (**segmenti**)
 - Ogni segmento occupa **64KB**
 - Una locazione di memoria è individuata dall'indirizzo **segmento:offset**
 - segmento = numero del blocco da 64KB
 - offset = spiazzamento all'interno del segmento

22

Organizzazione della memoria (8086)

- La segmentazione prevede l'uso di due componenti per specificare una locazione di memoria
 - un valore di **segmento**
 - un valore di spostamento (**offset**) all'interno del segmento
- Accesso alla locazione di memoria specificato dal segmento (**segment**) e dallo spostamento (**offset**)



23

Organizzazione della memoria (8086)

- L'**indirizzo fisico** (20 bit) di una locazione di memoria è calcolato utilizzando due registri a 16 bit
 - Un registro contiene il numero di segmento e un altro registro contiene l'offset
- I valori dei due registri costituiscono l'**indirizzo logico** (indirizzo **segmentato**) della locazione
- Ogni registro di segmento punta alla base del proprio segmento
 - 0000**0**h, 0001**0**h, 0002**0**h, ..., 000F**0**h, 0010**0**h, ..., FFFF**0**h (indirizzi espressi in esadecimale)
 - Ultima cifra sempre a zero (valori multipli di sedici) → può essere eliminata nella rappresentazione dell'indirizzo

24

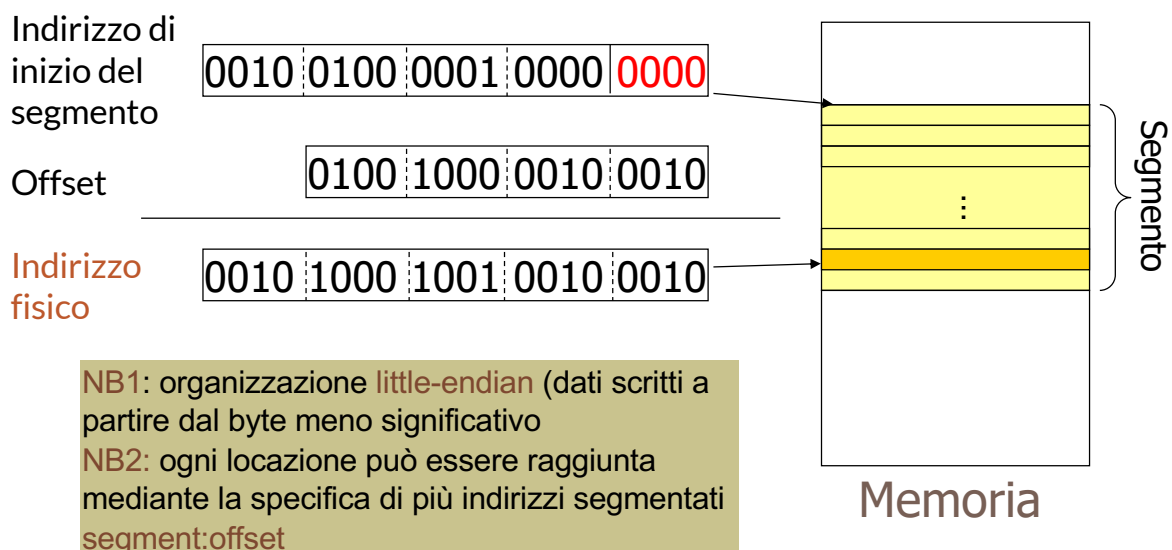
Organizzazione della memoria (8086)

- I progettisti del processore ritennero **sufficienti** 1MB di memoria e l'impiego di registri a 16 bit
- Il processore 8086 ha registri a 16 bit
 - $2^{16} = 65536 = 64K$
- Segment da 16 bit → 65536 diversi segmenti
- Offset da 16 bit → un segmento non può superare 64Kbyte
- Registri interni alla CPU: **16 bit**

25

Organizzazione della memoria (8086)

- Funzione per convertire l'indirizzo **segmentato** in indirizzo **fisico**



26

Organizzazione dei registri (8086)

- La CPU INTEL 8086 possiede 14 registri visibili all'utente, ciascuno di 16 bit
 - 4 registri generali
 - 4 registri puntatori o indici
 - 4 registri segmento
 - 2 registri di controllo
 - contatore di programma
 - registro di flag

28

Registri generali (8086)

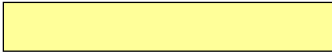
AX	7 0 7 0 15 0 AH AL	Registro accumulatore Può essere usato in maniera implicita come accumulatore nelle operazioni aritmetiche
BX	BH BL	Registro base Può essere usato per l'indirizzamento di una locazione di memoria
CX	CH CL	Registro contatore Può essere usato come contatore nei cicli e nelle istruzioni di shift e rotazione
DX	DH DL High Low	Registro dati Può essere usato come ampliamento dell'accumulatore nelle operazioni aritmetiche

29

Organizzazione dei registri (8086)

Registri indice

SI 

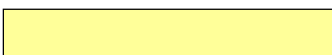
DI 

Source Index e Destination Index

Sono spesso usati nelle istruzioni che operano su stringhe, per referenziare le locazioni estreme della zona di memoria contenente la stringa

Registri puntatori

SP 

BP 

Stack Pointer

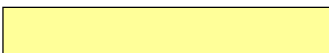
È il puntatore al top dello stack

Stack Base Pointer

È il puntatore alla base dello stack

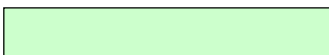
30

Organizzazione dei registri (8086)

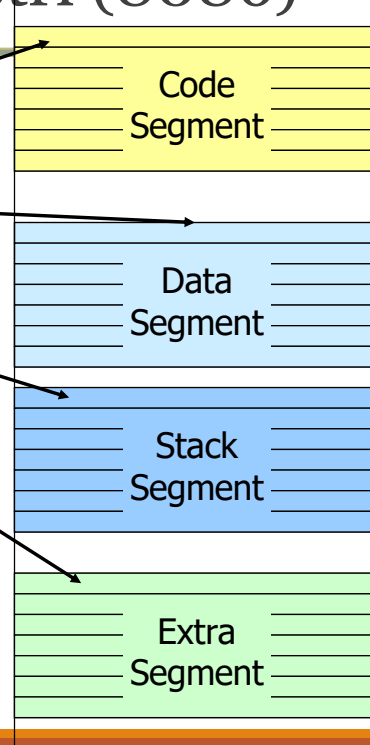
CS 

DS 

SS 

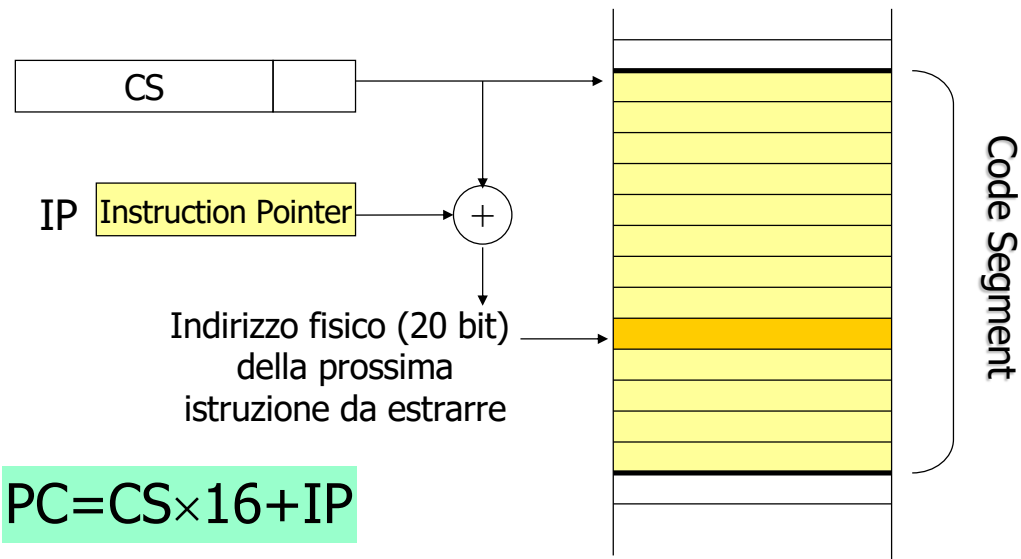
ES 

I registri di segmento puntano ai segmenti correntemente **attivi**. Contengono un valore che, **moltiplicato per 16**, fornisce un indirizzo fisico a 20 bit che rappresenta l'**indirizzo di testa** dei segmenti usati dal programma.



31

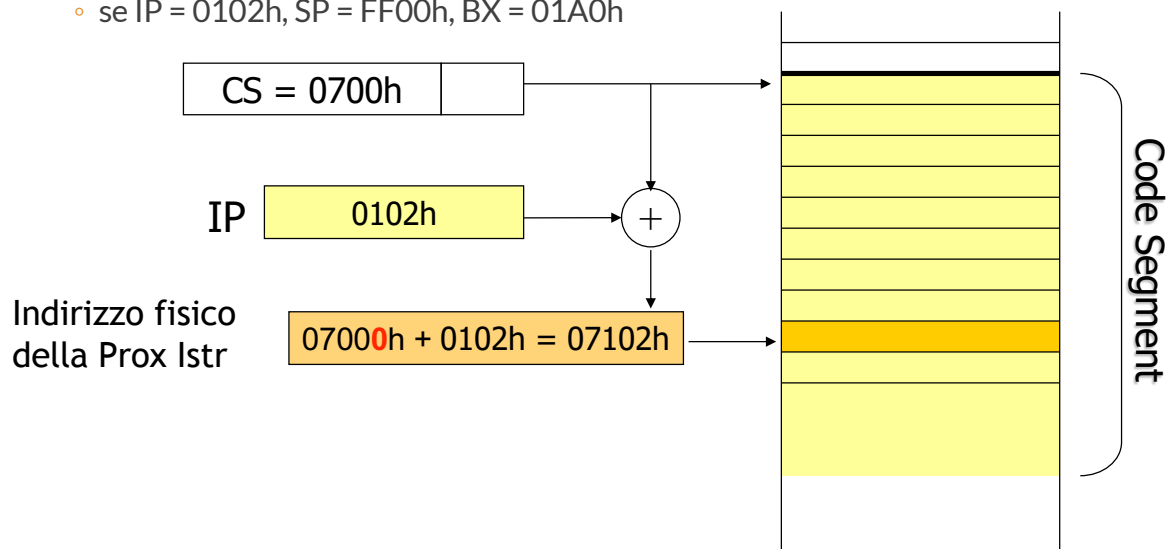
Organizzazione dei registri (8086)



32

Organizzazione dei registri (8086)

- Esempio: Qual è l'indirizzo fisico della prossima istruzione?
 - se CS = DS = SS = 0700h (unico seg)
 - se IP = 0102h, SP = FF00h, BX = 01A0h

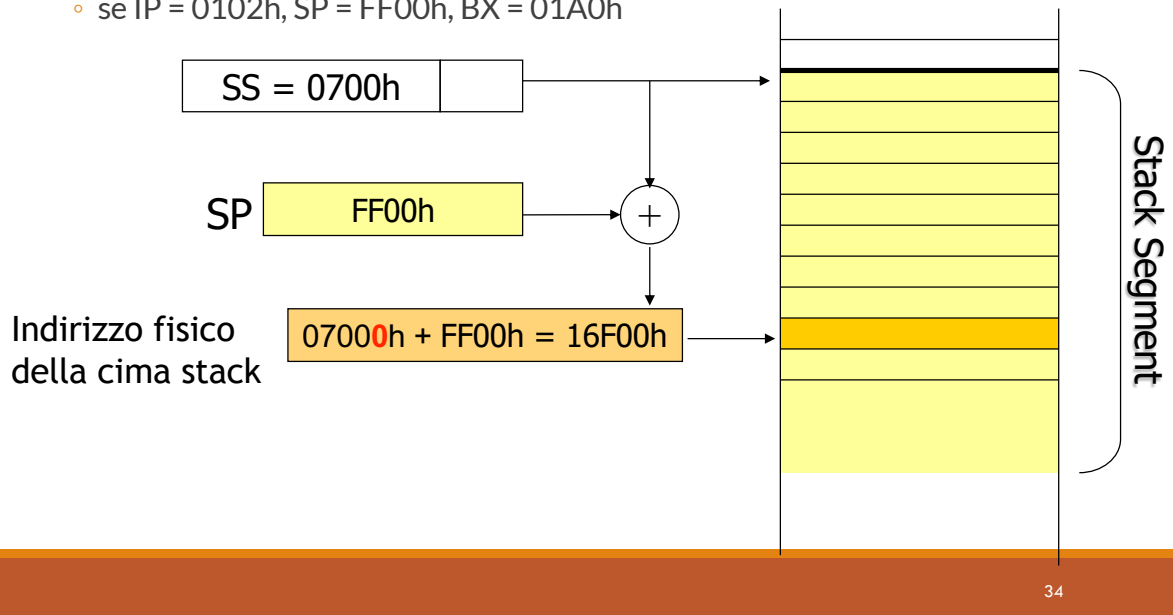


33

Organizzazione dei registri (8086)

- Esempio: Qual è l'indirizzo fisico del top dello stack?

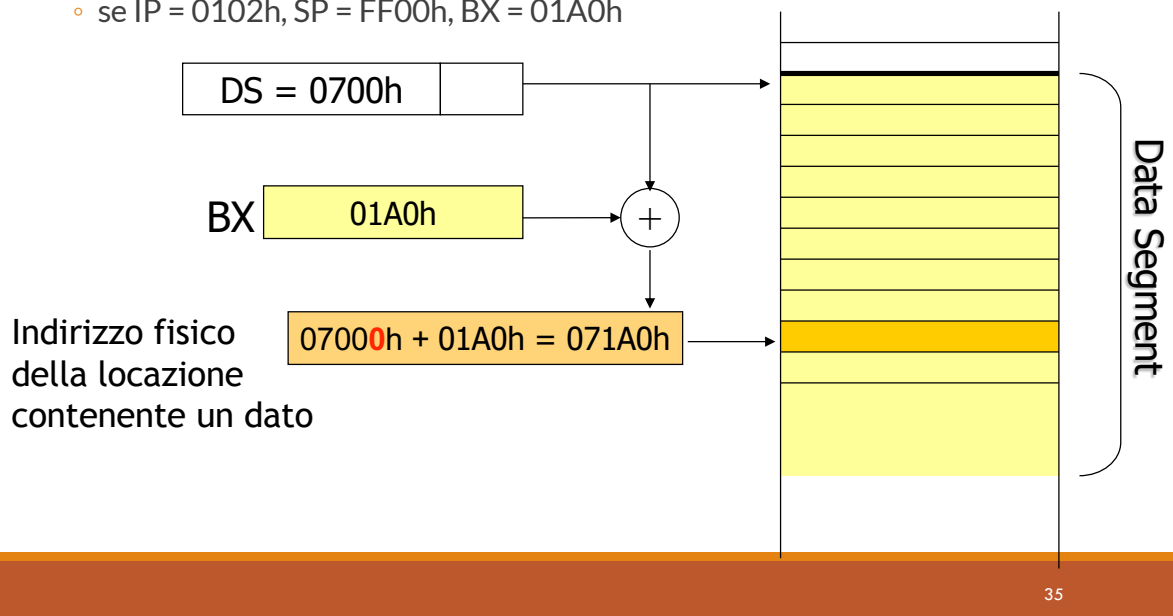
- se CS = DS = SS = 0700h (unico seg)
- se IP = 0102h, SP = FF00h, BX = 01A0h



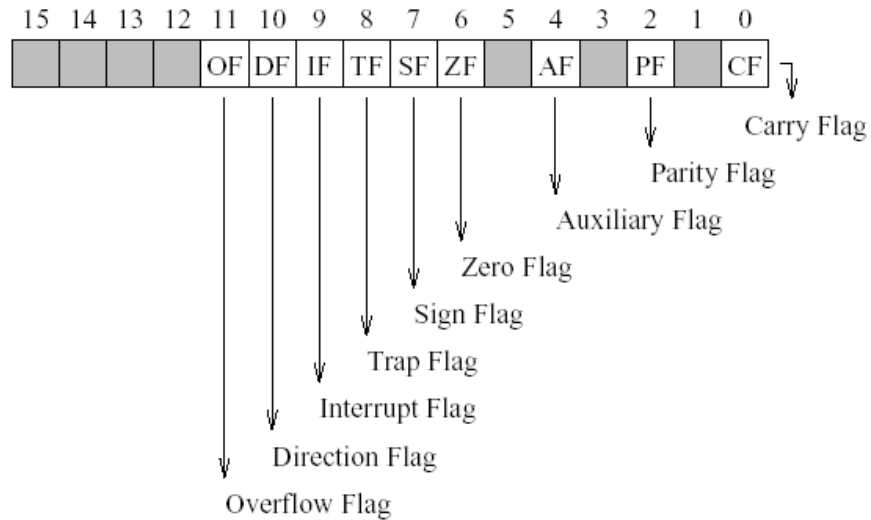
Organizzazione dei registri (8086)

- Esempio: Qual è l'indirizzo fisico di un dato in memoria?

- se CS = DS = SS = 0700h (unico seg)
- se IP = 0102h, SP = FF00h, BX = 01A0h



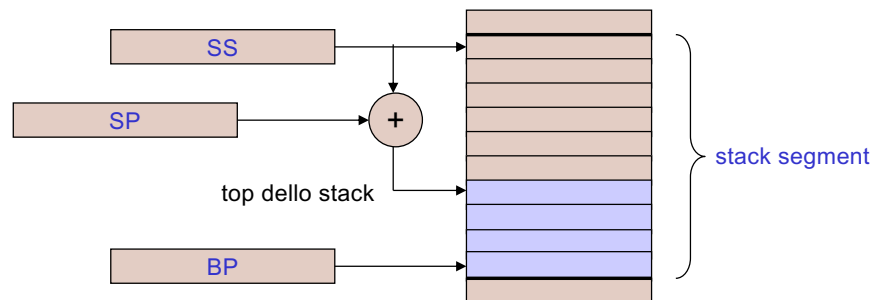
Organizzazione dei registri (8086)



37

Organizzazione dello stack (8086)

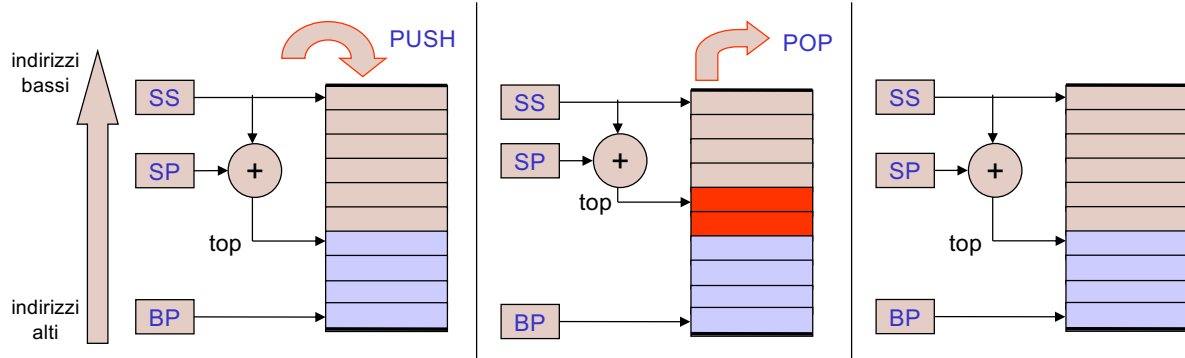
- All'interno dello stack non si stabiliscono indirizzi assoluti per le singole variabili
 - riferimento ai dati di tipo implicito
- Un registro punta all'inizio del segmento dello stack (**SS**)
- Un registro punta al top dello stack (**SP**, offset da SS)
- Un registro punta alla base dello stack (**BP**)



38

Organizzazione dello stack (8086)

- L'operazione di **PUSH** permette di inserire una parola in cima allo stack
 - al termine della PUSH il registro SP è **decrementato di 2** ($SP=SP-2$)
- L'operazione di **POP** permette di prelevare una parola dalla cima allo stack
 - al termine della POP il registro SP è **incrementato di 2** ($SP=SP+2$)



39

Panoramica del livello ISA del Pentium 4

Registri principali di Pentium 4

Bits		16	8	8	
	AH	A	X	AL	EAX
	BH	B	X	BL	EBX
	CH	C	X	CL	ECX
	DH	D	X	DL	EDX

	ESI
	EDI
	EBP
	ESP

	CS
	SS
	DS
	ES
	FS
	GS

	EIP
--	-----

	EFLAGS
--	--------

Livello ISA dell'UltraSPARC III

- Architettura a 64 bit
 - Registri a 64 bit
 - Indirizzamento di $2^{64} = 16$ ExaByte (potenziale, effettivo: 16 TByte)
- Modello di memoria big endian (commutabile in little endian)
- 64 registri
 - 32 registri generali + 32 registri in virgola mobile
 - Registro speciale R0, costante a zero
 - Organizzazione dei registri a “finestre”
- Architettura load/store